

Lead technique - TP Queuing

Le but général du TP est d'autoriser du background processing. Ce qui va être fait sera une itération de l'app actuelle (permettant de lister des photos de flickr), l'objectif étant de zipper les 10 premières photos du résultat de recherche et de les mettre à disposition de l'utilisateur

I. Ajouter un endpoint et un bouton appelant le endpoint

Dans le but de zipper le résultat, un bouton doit être ajouté sur la vue d'un résultat d'une liste.

Ce bouton appellera un nouvel endpoint POST /zip avec des query params "tags". Créer le endpoint et récupérer les tags, la logique du endpoint est au point II

II. Endpoint producer

Créer le code pour envoyer dans une queue les tags demandés à zipper.

Afin de s'authentifier sur le projet Google qui contient le pubsub (queuing server), un fichier de clef en JSON d'un service account vous a été fourni. Cette clef doit être mise quelque part dans votre file system mais **PAS** dans votre repository github. Seul le path absolu de ce fichier sera mis dans une variable d'environnement nommée GOOGLE_APPLICATION_CREDENTIALS. ([Cf.](#))

Pour instaurer cette variable d'environnement, vous pouvez vous aider de la librairie [dotenv](#). Pour la mise en production de votre projet, il est possible d'uploader des secret files et de mettre des variables d'environnement directement depuis l'UI de render.com

Pour la librairie permettant de produire des items, utiliser [celle-ci](#) (Publish message). Cette librairie utilisera automatiquement la variable d'environnement créée précédemment pour se connecter à Google Cloud.

Le topic et la souscription à utiliser ont été créés avec un nombre qui vous sera attribué sous la forme "ecni2-" + i. (i = le nombre attribué)

Le project id Google Cloud est "ecni2-2026"

La forme du message envoyé importe peu, vous le recevrez finalement de la même manière du côté worker.

III. Worker - Ecoute des messages

Dans le but de se simplifier la tâche et d'éviter d'avoir 50 instances de serveur sur render.com, nous utiliserons l'instance de l'API, en production et dans des conditions réelles, cela ne devrait pas fonctionner ainsi et devrait être séparé dans un autre serveur.

Dans le fichier server.js, faites un require/import d'un fichier js que vous créerez pour consommer les messages de la queue.

Pour consommer les messages de la queue, référez-vous à ce [lien](#) (Listen for messages), en utilisant encore une fois “ecni2-” + i pour le topic et la subscription.

IV. Worker - Fetch des images - zippage - acknowledgment

Le but est maintenant de zipper les fichiers reliés aux tags que vous avez reçu dans votre message.

Le bucket est : ecni22026bucket

Appelez d’abord flickr pour obtenir les urls des images correspondantes, prenez les 10 premières et zipper les en utilisant [ceci](#).

Stockez votre zip dans Google Cloud Storage en utilisant ce snippet avec un filename aléatoire:

```
import { Storage } from '@google-cloud/storage';
let storage = new Storage();
const file = await storage
  .bucket(profileBucket)
  .file('public/users/' + filename);
const stream = file.createWriteStream({
  metadata: {
    contentType: uploadedFile.mimetype,
    cacheControl: 'private'
  },
  resumable: false
});

return new Promise ((resolve, reject) => {
  stream.on('error', (err) => {
    reject(err);
  });

  stream.on('finish', () => {
    resolve('Ok');
  });
  stream.end(uploadedFile.buffer);
});
```

A ce moment-là, il faudrait normalement stocker dans une BDD l’état successful de votre “job” (clef: nos tags, valeur: le lien vers le fichier). Nous n’allons pas utiliser de BDD pour cela dans ce cas d’expérimentation pour accélérer le développement. En sachant que le worker est sur la même instance que l’API, stockez simplement que votre job s’est bien terminé dans une variable static/globale.

V. Récupération du lien du fichier téléchargé

Lors de l’appel au endpoint GET /, si vous avez déjà les tags, utilisez le snippet ci-dessous pour générer un lien de téléchargement du zip que vous insérerez dans votre page générée.

```
const options = {
  action: 'read',
```

```
    expires: moment().add(2, 'days').unix() * 1000
  };
  const signedUrls = await storage
    .bucket(process.env.STORAGE_BUCKET)
    .file(name)
    .getSignedUrl(options);
```